

```

medikondarakesh@Medikondas-MacBook-Air ~ % nano multi.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall multi.c -o multi
medikondarakesh@Medikondas-MacBook-Air ~ % ./multi

MultiPipe Shell > ls -la | grep multi
-rwxr-xr-x    1 medikondarakesh  staff   34680  2 Oct 14:25 multi
-rw-r--r--    1 medikondarakesh  staff    3986  2 Oct 14:22 multi_pipe.c
-rw-r--r--    1 medikondarakesh  staff    2700  2 Oct 14:24 multi.c
drwxr-xr-x    3 medikondarakesh  staff     96  2 Oct 14:25 multi.dSYM

MultiPipe Shell > cat /etc/passwd | cut -d: -f1 | head -n 5
##
# User Database
#
# Note that this file is consulted directly only when the system is running
# in single-user mode.  At other times this information is provided by

MultiPipe Shell > exit
medikondarakesh@Medikondas-MacBook-Air ~ %

```

```

~ --zsh    ...    ...ory_and_test_shell.c    ...ano signal_handler.c    ~ --zsh    ~ -- append_stderr
UW PICO 5.09    File: multi_pipe.c

```

```

// Parse commands delimited by '|' [cite: 5]
char *token = strtok(line, "|"); [cite: 5]
while (token != NULL && num_cmds < MAX_CMDS) { [cite: 5]
    cmds[num_cmds++] = trim(token); [cite: 5]
    token = strtok(NULL, "|"); [cite: 5]
}
if (num_cmds == 0) return; [cite: 5]

// Allocate pipe file descriptors array: (N-1) pipes require 2*(N-1) descriptors [cite: 6]
int num_pipes = num_cmds - 1; [cite: 6]
int pipefds[2 * num_pipes]; [cite: 6]

for (int i = 0; i < num_pipes; i++) { [cite: 6]
    if (pipe(pipefds + i * 2) < 0) { [cite: 6]
        perror("pipe allocation failed"); [cite: 6]
        return; [cite: 6]
    }
}

pid_t pids[MAX_CMDS]; [cite: 6]
for (int i = 0; i < num_cmds; i++) { [cite: 6]
    // Tokenize current command into arguments [cite: 6]
    char *args[MAX_ARGS]; [cite: 6]
    int arg_count = 0; [cite: 6]
    char *arg_token = strtok(cmds[i], "\t"); [cite: 6]
    while (arg_token != NULL && arg_count < MAX_ARGS - 1) { [cite: 6]
        args[arg_count++] = arg_token; [cite: 6]
        arg_token = strtok(NULL, "\t"); [cite: 6]
    }
    args[arg_count] = NULL; [cite: 6]

    if (arg_count == 0) continue; [cite: 6]

    pids[i] = fork(); [cite: 6]
    if (pids[i] < 0) { [cite: 6]
        perror("fork failed"); [cite: 6]
        return; [cite: 6]
    } else if (pids[i] == 0) { [cite: 6]
        // Child Process: Redirect standard input if not first command [cite: 6]
        if (i != 0) { [cite: 6]
            if (dup2(pipefds[(i - 1) * 2], STDIN_FILENO) < 0) { [cite: 6]

```

UW PICO 5.09

File: multi_pipe.c

```

int arg_count = 0;[cite: 6]
char *arg_token = strtok(cmds[i], " \t");[cite: 6]
while (arg_token != NULL && arg_count < MAX_ARGS - 1) {[cite: 6]
    args[arg_count++] = arg_token;[cite: 6]
    arg_token = strtok(NULL, " \t");[cite: 6]
}
args[arg_count] = NULL;[cite: 6]

if (arg_count == 0) continue;[cite: 6]

pids[i] = fork();[cite: 6]
if (pids[i] < 0) {[cite: 6]
    perror("fork failed");[cite: 6]
    return;[cite: 6]
} else if (pids[i] == 0) {[cite: 6]
    // Child Process: Redirect standard input if not first command[cite: 6]
    if (i != 0) {[cite: 6]
        if (dup2(pipefds[i - 1] * 2, STDIN_FILENO) < 0) {[cite: 6]
            perror("dup2 stdin failed");[cite: 6]
            exit(EXIT_FAILURE);[cite: 7]
        }
    }

    // Child Process: Redirect standard output if not last command[cite: 7]
    if (i != num_cmds - 1) {[cite: 7]
        if (dup2(pipefds[i * 2 + 1], STDOUT_FILENO) < 0) {[cite: 7]
            perror("dup2 stdout failed");[cite: 7]
            exit(EXIT_FAILURE);[cite: 7]
        }
    }

    // Close all pipe descriptors inside child process[cite: 7]
    for (int j = 0; j < 2 * num_pipes; j++) {[cite: 7]
        close(pipefds[j]);[cite: 7]
    }

    execvp(args[0], args);[cite: 7]
    perror("execvp failed");[cite: 7]
    exit(EXIT_FAILURE);[cite: 7]
}

// Parent Process: Close all pipe descriptors to send EOF properly[cite: 7]
for (int i = 0; i < 2 * num_pipes; i++) {[cite: 7]
    close(pipefds[i]);[cite: 7]
}

// Synchronize and monitor termination of all launched children[cite: 7]

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

UW PICO 5.09

File: multi_pipe.c

```

    if (i != num_cmds - 1) {[cite: 7]
        if (dup2(pipefds[i * 2 + 1], STDOUT_FILENO) < 0) {[cite: 7]
            perror("dup2 stdout failed");[cite: 7]
            exit(EXIT_FAILURE);[cite: 7]
        }
    }

    // Close all pipe descriptors inside child process[cite: 7]
    for (int j = 0; j < 2 * num_pipes; j++) {[cite: 7]
        close(pipefds[j]);[cite: 7]
    }

    execvp(args[0], args);[cite: 7]
    perror("execvp failed");[cite: 7]
    exit(EXIT_FAILURE);[cite: 7]
}

// Parent Process: Close all pipe descriptors to send EOF properly[cite: 7]
for (int i = 0; i < 2 * num_pipes; i++) {[cite: 7]
    close(pipefds[i]);[cite: 7]
}

// Synchronize and monitor termination of all launched children[cite: 7]
for (int i = 0; i < num_cmds; i++) {[cite: 7]
    int status;[cite: 7]
    waitpid(pids[i], &status, 0);[cite: 7]
}

}

int main() {[cite: 7]
    char input_buf[BUF_SIZE];[cite: 7]
    while (1) {[cite: 7]
        printf("\nMultiPipe Shell > ");[cite: 7]
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;[cite: 7]
        input_buf[strcspn(input_buf, "\n")] = 0; // Remove newline[cite: 7]
        if (strcmp(input_buf, "exit") == 0) break;[cite: 7]
        if (strlen(input_buf) == 0) continue;[cite: 7]
        execute_pipeline(input_buf);[cite: 7]
    }
    return 0;[cite: 7]
}

```

UW PICO 5.09

File: multi_pipe.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_CMDS 16
#define MAX_ARGS 32
#define BUF_SIZE 1024

// Trim whitespace from string ends[cite: 5]
char *trim(char *str) {
    while (*str == ' ' || *str == '\t') str++;[cite: 5]
    if (*str == 0) return str;[cite: 5]
    char *end = str + strlen(str) - 1;[cite: 5]
    while (end > str && (*end == ' ' || *end == '\t')) end--;[cite: 5]
    end[1] = '\0';[cite: 5]
    return str;[cite: 5]
}

// Execute arbitrary-length multi-pipe command pipelines[cite: 5]
void execute_pipeline(char *line) {
    char *cmds[MAX_CMDS];[cite: 5]
    int num_cmds = 0;[cite: 5]

    // Parse commands delimited by '|' [cite: 5]
    char *token = strtok(line, "|");[cite: 5]
    while (token != NULL && num_cmds < MAX_CMDS) {[cite: 5]
        cmds[num_cmds++] = trim(token);[cite: 5]
        token = strtok(NULL, "|");[cite: 5]
    }
    if (num_cmds == 0) return;[cite: 5]

    // Allocate pipe file descriptors array: (N-1) pipes require 2*(N-1) descriptors[cite: 6]
    int num_pipes = num_cmds - 1;[cite: 6]
    int pipefds[2 * num_pipes];[cite: 6]

    for (int i = 0; i < num_pipes; i++) {[cite: 6]
        if (pipe(pipefds + i * 2) < 0) {[cite: 6]
            perror("pipe allocation failed");[cite: 6]
            return;[cite: 6]
        }
    }

    pid_t pids[MAX_CMDS];[cite: 6]
    for (int i = 0; i < num_cmds; i++) {[cite: 6]
        // Tokenize current command into arguments[cite: 6]
        char *args[MAX_ARGS];[cite: 6]
```

 Get Help
 Exit

 WriteOut
 Justify

 Read File
 Where is

 Prev Pg
 Next Pg

 Cut Text
 UnCut Tex